

LuxCipher Implementation Notes

Implementation choices and current architecture

This document tracks the implementation choices made while building LuxCipher. It is the source for the PDF reference document and should be updated whenever the project changes in a meaningful way.

Project Direction

LuxCipher is being built as a desktop password manager first. Web features can be added later, but the current direction is a local desktop application with security-critical logic kept separate from the interface.

The project is intentionally growing in small steps. The current scope is:

- a secure password generator;
- a first decrypted vault data model;
- a local-only account authentication core;
- documentation for security decisions and future work.

The project is not yet a complete password manager. It does not yet save passwords, encrypt a vault file, or unlock a vault UI.

Branch Policy

Development work is done on the `develop` branch.

Current Structure

- `luxcipher/password_generator.py` contains password generation logic.
- `luxcipher/desktop_app.py` contains the Tkinter desktop interface.
- `luxcipher/vault_model.py` contains the decrypted vault data model.
- `luxcipher/auth.py` contains local account authentication logic.
- `luxcipher/__main__.py` allows the app to run with `python -m luxcipher`.
- `tests/` contains automatic tests for core behavior.
- `docs/SECURITY.md` records security rules.
- `docs/ROADMAP.md` records the incremental build plan.
- `docs/IMPLEMENTATION_NOTES.md` records implementation choices.
- `output/pdf/luxcipher-implementation-notes.pdf` is the generated PDF version of these notes.
- `tools/build_implementation_notes_pdf.py` regenerates the PDF from this Markdown file.
- `.gitattributes` marks PDF and PNG artifacts as binary so Git does not apply text line-ending normalization to generated visual files.

Password Generator

The first implemented feature is a secure password generator. It lives in `luxcipher/password_generator.py`.

The generator uses Python's `secrets` module instead of `random`. This matters because `random` is designed for simulations and general-purpose randomness, while `secrets` is designed for passwords, tokens, and other security-sensitive values.

The generator accepts a `PasswordOptions` dataclass with these settings:

- `length`, default 20;
- `use_lowercase`;
- `use_uppercase`;
- `use_digits`;
- `use_symbols`;
- `exclude_ambiguous`.

`PasswordOptions` is frozen, so once an options object is created, it cannot be modified accidentally.

The minimum password length is 12 characters. The UI and the generation logic both use the same `MIN_PASSWORD_LENGTH` constant, so the rule is enforced even if the generator is reused later outside the desktop app.

The generator builds one character pool for each enabled option:

- lowercase letters;
- uppercase letters;
- digits;
- symbols.

If `exclude_ambiguous` is enabled, visually confusing characters such as 0, O, I, l, and 1 are removed from the pools.

The generation algorithm works like this:

- 1 Build the enabled character pools.
- 2 Reject lengths under the secure minimum.
- 3 Reject a request with no enabled character set.
- 4 Pick at least one character from each enabled set.
- 5 Fill the remaining length from all enabled characters.
- 6 Shuffle the result with `secrets.SystemRandom().shuffle`.

This ensures that if the user enables a set, the final password contains at least one character from that set. Security still primarily comes from length and cryptographically secure randomness, not from composition rules alone.

Desktop Interface

The desktop app lives in `luxcipher/desktop_app.py` and uses Tkinter. Tkinter was chosen because it is part of the Python standard library, lets the project start as a real desktop application, and avoids introducing frontend or web dependencies too early.

The UI currently provides:

- a read-only generated password field;
- a length selector;
- checkboxes for lowercase, uppercase, digits, symbols, and ambiguous character exclusion;
- a Generate button;
- a Copy button;
- a status label.

The UI does not contain password-generation rules directly. It reads the selected values, creates a `PasswordOptions` object, and delegates generation to `generate_password`. This keeps the security-sensitive logic testable without opening a graphical window.

The Copy button uses the system clipboard through Tkinter. This is convenient, but clipboard handling is listed as a future security consideration because other applications may be able to read clipboard contents.

Vault Data Model

The first roadmap point is the vault shape. This is implemented in `luxcipher/vault_model.py`.

The model represents decrypted data while the vault is unlocked in memory. It does not represent the encrypted on-disk file yet.

`VaultEntry` represents one password record and currently contains:

- `id`;
- `title`;
- `username`;
- `password`;
- `url`;
- `notes`;
- `created_at`;
- `updated_at`.

The `id` is generated with UUID4. It is meant to be a stable identifier for a record, independent from title or username. This matters because titles can change and are not guaranteed to be unique.

`title` and `password` are required. `username`, `url`, and `notes` are optional strings.

Timestamps are timezone-aware UTC datetimes. This avoids ambiguous local-time data and gives future syncing or audit features a cleaner base.

`VaultData` represents the decrypted vault payload and currently contains:

- `schema_version`;
- `created_at`;
- `updated_at`;
- `entries`.

The schema version starts at 1. Unsupported schema versions are rejected so future migrations can be explicit instead of silently misreading data.

Both `VaultEntry` and `VaultData` can be converted to and from dictionaries. This is not persistence yet. It only defines a stable shape that can later be encoded as JSON bytes and encrypted.

Local Authentication

The next implemented foundation is local-only authentication. It lives in `luxcipher/auth.py`.

The goal is to support a local account and master password without adding a remote server or cloud identity system. This maximizes local control: account metadata stays on the user's device, and LuxCipher does not need a backend to authenticate the user.

The current local account model is `LocalAccount`. It contains:

- `schema_version`;
- `id`;
- `username`;
- `created_at`;
- `updated_at`;
- `kdf`;
- `password_verifier`.

The local account deliberately does not store:

- the master password;
- the derived master key;
- decrypted vault data;
- vault entries.

The username is a local handle, not an online identity. It must be 3-64 characters and can contain letters, numbers, `.`, `_`, and `-`. This keeps the first version simple and avoids confusing a local account with an email login or cloud account.

The master password must currently be at least 12 characters. This is a minimum validation rule, not a complete password-strength meter. Because a local attacker who steals the account metadata can try guesses offline, a long and memorable master password is still important.

Master Password Verification

LuxCipher verifies the master password without storing it.

The current design uses Python's standard `hashlib.scrypt` function as the key derivation function. The default parameters are:

- `n = 2**14`;
- `r = 8`;
- `p = 1`;
- `dklen = 32`;
- `maxmem = 64 MiB`;
- random 16-byte salt.

The salt and KDF parameters are public and stored with the local account record. This is expected: the salt is not a secret. Its job is to make identical master passwords produce different derived keys across

accounts.

KDF parameters loaded from serialized account metadata are validated before use. The implementation rejects unsupported KDF names, malformed base64 salts, and scrypt parameters above the allowed bounds. This prevents a corrupted or tampered local account record from asking the app to run unreasonable derivation settings.

The verifier is created in two steps:

- 1 Derive a key from the master password with scrypt.
- 2 Compute HMAC-SHA256 over the context string

`LuxCipher local account verifier v1.`

The result is stored as `password_verifier`.

Authentication recomputes the verifier from the candidate password and compares it with the stored verifier using `hmac.compare_digest`. This avoids regular string comparison for sensitive verification values.

This design prevents accidental storage of the master password, but it is not magic. If an attacker obtains the local account metadata, they can still run an offline guessing attack. Scrypt slows that attack down, and the random salt prevents simple precomputed table reuse, but the user still needs a strong master password.

The current verifier is only for account authentication. Vault encryption key derivation is intentionally not implemented yet. Later, LuxCipher should derive separate context-specific keys so the authentication verifier and vault encryption do not reuse one raw key directly.

Encryption Boundary

The current model is plaintext by design because it represents data after the vault has been unlocked. When storage is added, the entries should not be stored directly in this form.

The expected future boundary is:

- decrypted `VaultData` exists only in memory while unlocked;
- serialized vault payload is encrypted before writing to disk;
- the master password is never stored;
- only public encryption metadata should remain outside the ciphertext.

Examples of public metadata that may be stored unencrypted later:

- file format version;
- KDF name;
- KDF salt;
- KDF parameters;
- encryption algorithm name;
- nonce or IV.

Entry fields such as title, username, password, URL, and notes should be inside the encrypted payload.

Tests

The password generator tests verify:

- generated password length;
- inclusion of each enabled character set;
- exclusion of ambiguous characters;
- rejection of no selected character sets;
- rejection of too-short passwords;
- acceptance of the minimum valid length.

The vault model tests verify:

- UUID creation;
- required title and password;
- timezone-aware timestamps;
- entry dictionary round-trip;
- empty vault defaults;
- vault dictionary round-trip;
- schema version rejection;
- rejection of impossible timestamp ordering.

The local authentication tests verify:

- local account creation;
- correct master password verification;
- wrong master password rejection;
- absence of the master password from serialized account metadata;
- dictionary round-trip;
- random salt and verifier differences for the same password;
- weak master password rejection;
- invalid username rejection;
- account schema version rejection;
- unsupported KDF rejection;
- tampered scrypt parameter rejection;
- invalid base64 salt rejection.

These tests focus on core logic rather than the GUI. This is intentional because the core logic should remain testable independently from the desktop window.

Current Limitations

LuxCipher is still early. The current implementation does not yet include:

- encrypted vault storage;
- deriving a vault encryption key;
- unlock and lock workflows;
- create, read, update, and delete workflows in the UI;

- clipboard timeout or automatic clipboard clearing;
- desktop packaging.

These are future steps, not hidden features.

Recruiter Explanation

A concise way to describe the current project:

I am building LuxCipher as a desktop password manager in small, testable steps. I started with a secure password generator using Python's `secrets` module and separated the generation logic from the Tkinter UI. Then I defined a plaintext in-memory vault model with UUID-based entries, required fields, UTC timestamps, schema versioning, and dictionary serialization. I then added local-only account authentication using script-derived verification, without storing the master password. I have not implemented vault storage or encryption yet because I want the account model, data model, and tests to be clear before adding authenticated encryption.

If asked why `secrets` is used:

`secrets` is designed for security-sensitive randomness, while `random` is not. Password generation needs unpredictability, so `secrets.choice` and `secrets.SystemRandom` are the safer standard-library choice.

If asked whether this is already a password manager:

Not yet. It is the foundation of one: password generation, local account authentication, and the first vault data model. The next major step is deriving a vault encryption key and adding encrypted local storage.

If asked what security principle guided the design:

I am keeping cryptographic decisions conservative: no custom crypto, no stored master password, authenticated encryption later, and sensitive entry fields encrypted as one vault payload.

If asked why accounts are local:

A password manager does not need a server to authenticate a local vault. A local account avoids backend risk, cloud recovery complexity, and remote identity dependencies. The tradeoff is that recovery becomes the user's responsibility, so the master password and local backup strategy matter more.